UD3 — Diseño y realización de pruebas

Entornos de Desarrollo · 1.º DAW · 15 horas

Índice

1 Antes de empezar **2**
1.1 Qué vas a saber hacer al terminar esta unidad . . . 2

2 Por qué se prueba **3**
2.1 El coste del error . . . 3
2.2 Casos reales que conviene conocer . . . 3
2.3 Verificación vs validación . . . 3
2.4 Error, defecto, fallo . . . 3

3 Tipos de pruebas **4**
3.1 Por conocimiento del código . . . 4
3.2 Por nivel (la pirámide de pruebas) . . . 4
3.3 Otros tipos que hay que saber nombrar . . . 4
3.4 Manuales vs automáticas . . . 5

4 Diseño de casos de prueba **6**
4.1 Qué es un caso de prueba . . . 6
4.2 Técnica 1: Clases de equivalencia (caja negra) . . . 6
4.3 Técnica 2: Valores límite (caja negra) . . . 6
4.4 Técnica 3: Tabla de decisión (caja negra) . . . 7
4.5 Técnica 4: Cobertura (caja blanca) . . . 7

5 Documentar las pruebas **9**
5.1 Plan de pruebas . . . 9
5.2 Informe de incidencia (*bug report*) . . . 9

6 Depuración **11**
6.1 Qué es depurar . . . 11
6.2 El método (antes de tocar el depurador) . . . 11
6.3 `console.log` está bien… hasta cierto punto . . . 11
6.4 El depurador de VS Code . . . 11

7 Pruebas unitarias automáticas con Jest **13**
7.1 Instalación . . . 13
7.2 Anatomía de una prueba . . . 13
7.3 Matchers principales . . . 14
7.4 Preparación y limpieza . . . 14
7.5 Dobles de prueba (mocks) . . . 15
7.6 Cobertura . . . 15

8 Desarrollo dirigido por pruebas (TDD) 17
8.1 El ciclo rojo-verde-refactor . . . 17
8.2 Ejemplo: contador de tareas pendientes, con TDD . . . 17
8.3 Por qué funciona . . . 17

9 Calidad del software: normas 18
9.1 ISO/IEC 25010 — Modelo de calidad del producto . . . 18
9.2 Otras normas y certificaciones que conviene nombrar . . . 18

10 Resumen de la unidad 20

11 Autoevaluación 20

1 Antes de empezar

1.1 Qué vas a saber hacer al terminar esta unidad

Cubre el **RA3: diseña y realiza pruebas para verificar el funcionamiento de aplicaciones**, que además es el resultado de aprendizaje **dualizable** del módulo: todos sus criterios pueden desarrollarse también en la empresa. Al acabarla tienes que ser capaz de:

- Distinguir los tipos de prueba y decir cuál toca en cada momento.
- Diseñar casos de prueba con técnicas sistemáticas, no a ojo.
- Usar el depurador del IDE con soltura.
- Escribir pruebas unitarias automáticas con Jest y medir la cobertura.
- Documentar un plan de pruebas y un informe de incidencias.
- Aplicar criterios de calidad y conocer las normas de referencia.

Idea clave

Esta es la unidad que más te va a diferenciar en una entrevista de trabajo. Programar sabe mucha gente; escribir pruebas decentes, bastante menos. Y es lo primero que te van a pedir en cualquier empresa que se tome en serio su código.

2 Por qué se prueba

2.1 El coste del error

El coste de corregir un defecto crece de forma brutal según avanza el proyecto:

Fase en la que se detecta	Coste relativo aproximado
Requisitos	1×
Diseño	5×
Codificación	10×
Pruebas	15×
Producción	**50-150×**

Y eso solo contando el coste de arreglarlo. No cuenta la factura de un error en producción: datos corruptos, clientes perdidos, sanciones.

2.2 Casos reales que conviene conocer

- **Ariane 5, 1996**: el cohete explotó 37 segundos después del lanzamiento. Causa: se reutilizó código del Ariane 4 que convertía un número de 64 bits a 16 bits; los valores del Ariane 5 eran mayores y provocaron un desbordamiento. Coste: unos 370 millones de dólares. **No se probó el componente reutilizado con los datos del nuevo cohete.**
- **Mars Climate Orbiter, 1999**: un módulo trabajaba en unidades imperiales y otro en métricas. La sonda se desintegró en la atmósfera de Marte. 125 millones de dólares. **Fallo de prueba de integración.**
- **Knight Capital, 2012**: un despliegue dejó código antiguo activo en uno de ocho servidores. En 45 minutos la empresa perdió 440 millones de dólares y quebró. **Fallo de pruebas de despliegue.**

2.3 Verificación vs validación

Dos preguntas distintas que se confunden constantemente:

- **Verificación**: *¿estamos construyendo el producto **correctamente**?* ¿Cumple las especificaciones? Es una comprobación técnica interna.
- **Validación**: *¿estamos construyendo el producto **correcto**?* ¿Es lo que el cliente necesitaba? Es una comprobación con el usuario.

Un programa puede pasar toda la verificación y suspender la validación: hace exactamente lo que ponía el documento… y el documento estaba mal.

2.4 Error, defecto, fallo

Terminología estándar (ISTQB), que cae en los exámenes:

- **Error** (*error/mistake*): la equivocación humana. Alguien escribió < donde iba <=.
- **Defecto** (*defect/bug*): la consecuencia en el código. Esa línea mal escrita.
- **Fallo** (*failure*): lo que percibe el usuario cuando el defecto se ejecuta. «La última tarea no aparece en la lista».

Un defecto solo produce un fallo si se ejecuta la línea afectada con los datos adecuados. Por eso hay defectos que llevan años en producción sin que nadie lo note.

3 Tipos de pruebas

3.1 Por conocimiento del código

3.1.1 Caja negra (*black box*)

Se prueba **desde fuera**, sin mirar el código: se dan entradas y se comprueban salidas. Sirve para validar contra los requisitos.

- Ventaja: no te condiciona lo que hace el código; encuentras funcionalidad que falta.
- Inconveniente: puede dejar caminos del código sin ejecutar.

3.1.2 Caja blanca (*white box*, estructural)

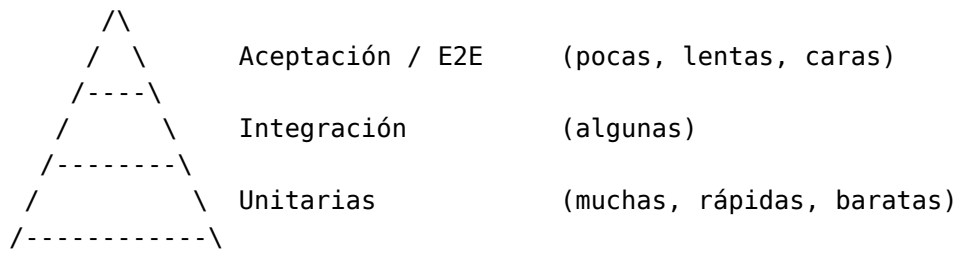
Se prueba **mirando el código**, diseñando casos que recorran sus caminos internos.

- Ventaja: permite medir y garantizar cobertura.
- Inconveniente: no detecta lo que **falta**; si olvidaste un requisito, no hay camino que probar.

3.1.3 Caja gris

Mezcla: conoces la estructura interna (por ejemplo, el esquema de la base de datos) pero pruebas por la interfaz externa.

3.2 Por nivel (la pirámide de pruebas)



Nivel	Qué prueba	Quién la hace	Herramienta típica
Unitaria	Una función o clase **aislada**	Quien programa	Jest, JUnit, PHPUnit
Integración	Que dos o más módulos encajan	Quien programa / QA	Jest, Supertest
Sistema	La aplicación completa en un entorno real	QA	Selenium, Cypress, Playwright
Aceptación	Que cumple lo que el cliente pidió	Cliente / usuario	Manual, Cucumber

Ojo

La pirámide se dibuja así por una razón económica: una prueba unitaria tarda milisegundos y señala exactamente la línea del problema; una prueba de extremo a extremo tarda segundos o minutos y solo te dice «algo ha fallado en alguna parte». Si tu proyecto tiene el triángulo al revés (muchas E2E, pocas unitarias), las pruebas tardarán tanto que dejaréis de ejecutarlas.

3.3 Otros tipos que hay que saber nombrar

Tipo	Qué comprueba
Regresión	Que lo que funcionaba **sigue** funcionando tras un cambio
Humo (*smoke*)	Un repaso rápido a lo esencial antes de probar en serio
Carga	Comportamiento con el volumen de uso previsto
Estrés	Dónde se rompe cuando se supera ese volumen
Rendimiento	Tiempos de respuesta
Usabilidad	Que una persona real sepa usarlo
Accesibilidad	Que sea usable con lector de pantalla, teclado, alto contraste
Seguridad	Inyección SQL, XSS, control de acceso
Compatibilidad	Distintos navegadores, dispositivos, sistemas
Alfa / Beta	En casa con usuarios reales / fuera con usuarios reales

3.4 Manuales vs automáticas

	Manuales	Automáticas
Coste inicial	Bajo	Alto (hay que programarlas)
Coste de repetir	Alto (una persona cada vez)	Casi cero
Fiabilidad	Humana: se cansa, se salta pasos	Siempre igual
Para qué son buenas	Usabilidad, exploración, casos nuevos	Regresión, unitarias, integración

La regla práctica: **si vas a ejecutar una prueba más de tres veces, automatízala.**

Práctica

P3.1 — Clasifica (45 min, en parejas)
Para cada situación, indica: (a) tipo de prueba por nivel, (b) caja blanca o negra, (c) si sería manual o automática, y (d) quién la haría.

1. Comprobar que `calcularIva(100, 'libro')` devuelve 104.
2. Comprobar que al pulsar «Guardar» en la web se crea la fila en la base de datos.
3. El cliente revisa si la aplicación hace lo que pidió antes de firmar la entrega.
4. Ver si la web aguanta 5.000 usuarios a la vez el día de la matrícula.
5. Ver a partir de cuántos usuarios se cae.
6. Después de arreglar un error, comprobar que no se ha roto nada más.
7. Ver si una persona mayor consigue pedir cita sin ayuda.
8. Comprobar que todas las ramas del `if` de `clasificarTarea` se ejecutan.
9. Comprobar que la aplicación se ve bien en Firefox, Chrome y Safari.
10. Comprobar que un usuario sin permisos no puede borrar tareas de otro.

4 Diseño de casos de prueba

Probar «a ver si funciona» no es probar. Hay técnicas sistemáticas que te dicen **qué casos elegir** y **cuántos** son suficientes.

4.1 Qué es un caso de prueba

Campo	Contenido
Identificador	CP-001
Descripción	Añadir tarea con título válido
Precondiciones	Existe `data/tareas.json` con 0 tareas
Datos de entrada	`añadir "Estudiar UD3"`
Pasos	1. Ejecutar el comando
Resultado esperado	Se crea una tarea con id 1, `hecha: false`, y se muestra «Tarea añadida»
Resultado obtenido	(se rellena al ejecutar)
Veredicto	Pasa / No pasa

4.2 Técnica 1: Clases de equivalencia (caja negra)

Idea: dividir el conjunto de entradas posibles en **clases** en las que el programa debería comportarse igual, y probar **un valor de cada clase**. Si funciona con uno, funcionará con los demás de esa clase.

Ejemplo: función `validarEdadAcceso(edad)` que acepta edades entre 18 y 65.

Clase	Tipo	Valor representativo
edad < 18	Inválida	10
18 ≤ edad ≤ 65	**Válida**	40
edad > 65	Inválida	80
No es un número	Inválida	`"hola"`
Ausente	Inválida	`undefined`

De infinitos valores posibles pasamos a **5 casos**.

4.3 Técnica 2: Valores límite (caja negra)

Los errores se concentran en las **fronteras**: el clásico < en lugar de <=. Se prueban el valor límite y sus vecinos inmediatos.

Para el rango [18, 65]:

Valor	Esperado	Por qué
17	Rechaza	Justo debajo del límite inferior
18	**Acepta**	Límite inferior
19	Acepta	Justo encima
64	Acepta	Justo debajo
65	**Acepta**	Límite superior
66	Rechaza	Justo encima del límite superior

Idea clave

Clases de equivalencia y valores límite **se usan juntas**: las clases te dan cobertura amplia con pocos casos, los límites cazan los errores más frecuentes. Es la combinación estándar en cualquier manual de pruebas.

4.4 Técnica 3: Tabla de decisión (caja negra)

Cuando la lógica depende de **varias condiciones combinadas**.

Ejemplo: se aplica descuento si el cliente es socio **y** la compra supera 50 €; si además es festivo, el descuento se dobla.

	R1	R2	R3	R4	R5	R6	R7	R8
¿Socio?	Sí	Sí	Sí	Sí	No	No	No	No
¿Compra > 50 €?	Sí	Sí	No	No	Sí	Sí	No	No
¿Festivo?	Sí	No	Sí	No	Sí	No	Sí	No
Descuento	20 %	10 %	0 %	0 %	0 %	0 %	0 %	0 %

Con 3 condiciones booleanas salen $2^3 = 8$ reglas. Se prueban todas, o se reducen las que claramente son equivalentes (R3 a R8 dan 0 %: basta probar dos o tres).

4.5 Técnica 4: Cobertura (caja blanca)

Mide qué porcentaje del código ejecutan tus pruebas.

Criterio	Qué exige	Ejemplo con `if (a && b)`
Cobertura de sentencias	Ejecutar cada línea al menos una vez	1 caso puede bastar
Cobertura de decisiones/ramas	Que cada `if` se evalúe a verdadero y a falso	2 casos
Cobertura de condiciones	Que cada condición simple sea V y F	a: V/F y b: V/F
Cobertura de caminos	Recorrer todas las combinaciones de caminos	Puede explotar

```
function clasificar(nota) {
  let r;
  if (nota >= 9)      r = 'sobresaliente';
  else if (nota >= 7) r = 'notable';
  else if (nota >= 5) r = 'aprobado';
  else                r = 'suspenso';
  return r;
}
```

Para **cobertura de decisiones** necesitas 4 casos: 9.5, 8, 6, 3. Añadiendo **valores límite**: 9, 8.99, 7, 6.99, 5, 4.99. Diez casos y esa función está blindada.

Ojo

Un 100 % de cobertura **no** significa que el programa esté bien. Significa que todas las líneas se han ejecutado, no que el resultado sea correcto ni que hayas pensado en todos los casos. Una prueba que ejecuta todo y no comprueba nada (`expect(true).toBe(true)`) da 100 % de cobertura y vale cero.

Práctica

P3.2 — Diseño sistemático de casos (2 h)
Función a probar: `validarDNI(cadena)` que devuelve `true` si es un DNI español válido. Reglas: 8 dígitos seguidos de una letra; la letra se obtiene con `"TRWAGMYFPDXBNJZSQVHLCKE"[numero % 23]`; se admite un guion opcional entre número y letra.

1. Identifica **todas** las clases de equivalencia (válidas e inválidas). Mínimo 8.
2. Identifica los **valores límite** relevantes.
3. Construye la **tabla de casos de prueba** completa, con el formato de 7 campos que hemos visto.
4. Calcula la **complejidad ciclomática** de una implementación razonable y comprueba si tu número de casos es coherente.
5. Implementa la función y ejecuta tus casos **a mano**, anotando resultado obtenido y veredicto.

(En la práctica P3.4 convertiremos esta tabla en pruebas automáticas de Jest. Guárdala.)

5 Documentar las pruebas

Criterio RA3-e y RA3-i. Se evalúa que las pruebas estén **planificadas y documentadas**, no solo ejecutadas.

5.1 Plan de pruebas

Documento que responde a: qué se prueba, cómo, cuándo, con qué y quién.

```
# Plan de pruebas — GestorTareas v1.0

## 1. Alcance
Se prueban los módulos `tarea.js`, `almacen.js` y `formato.js`.
Queda fuera: la interfaz de línea de comandos (se probará manualmente).

## 2. Estrategia
- Unitarias automáticas (Jest) para toda la lógica de negocio.
- Integración automática para la persistencia en fichero.
- Manuales exploratorias para la CLI.
- Objetivo de cobertura: ≥ 80 % de sentencias y ramas.

## 3. Entorno
Node 20, Jest 29, Windows 11 y Ubuntu 24.04.

## 4. Criterios de entrada
El código compila y `npm run lint` no da errores.

## 5. Criterios de salida (cuándo dejamos de probar)
- 100 % de los casos de prueba ejecutados.
- 0 incidencias de severidad crítica o alta abiertas.
- Cobertura ≥ 80 %.

## 6. Casos de prueba
(tabla o referencia a la hoja de casos)

## 7. Riesgos
La función de fecha depende del reloj del sistema: se inyecta `ahora` como parámetro.
```

5.2 Informe de incidencia (*bug report*)

Cuando una prueba falla, no basta con decir «no va». Un informe útil tiene:

Campo	Ejemplo
ID	INC-014
Título	Se pierden las tareas al añadir si no existe la carpeta `data`
Severidad	Alta (pérdida de datos)
Prioridad	Alta
Entorno	Node 20.11, Windows 11, versión 0.3.2
Pasos para reproducir	1. Borrar la carpeta `data`. 2. `npm start -- añadir "prueba"`
Resultado esperado	Se crea la carpeta y la tarea se guarda
Resultado obtenido	`Error: ENOENT: no such file or directory`
Evidencias	Captura, traza del error, fichero de registro
Asignado a	—
Estado	Abierta / En curso / Resuelta / Cerrada / Rechazada

Campo	Ejemplo

Idea clave

Severidad ≠ prioridad. Severidad es el daño técnico; prioridad es la urgencia de negocio. Una errata en el logotipo de la portada tiene severidad baja y prioridad altísima el día del lanzamiento. Un fallo que corrompe datos en un módulo que usa una persona al año tiene severidad crítica y prioridad media.

En la práctica esto se gestiona con herramientas: **GitHub Issues**, Jira, Redmine, Trello.

6 Depuración

6.1 Qué es depurar

Ejecutar el programa de forma controlada para ver qué hace por dentro. No es lo mismo que probar: las pruebas **detectan** que hay un defecto; el depurador te ayuda a **localizarlo**.

6.2 El método (antes de tocar el depurador)

1. **Reproduce** el fallo de forma fiable. Si no lo reproduces, no lo arreglas.
2. **Reduce** el caso al mínimo que sigue fallando.
3. **Formula una hipótesis** de dónde está el problema.
4. **Comprueba** la hipótesis con el depurador.
5. **Corrige**.
6. **Escribe una prueba** que falle con el bug y pase con la corrección. Así no vuelve.

Ojo

El punto 6 es el que separa a un profesional del resto. Cada bug arreglado debe dejar tras de sí una prueba de regresión. Si no, el mismo error volverá dentro de seis meses.

6.3 `console.log` está bien... hasta cierto punto

Sirve, es rápido, todo el mundo lo usa. Pero: ensucia el código, hay que recordar quitarlo, no te deja ver el estado completo y te obliga a reejecutar por cada cosa que quieras mirar.

Alternativas dentro del propio `console`:

```
console.table(tareas);            // tabla formateada, genial para arrays de objetos
console.group('Procesando');      // agrupa
console.error('algo va mal');     // sale por stderr y en rojo
console.time('carga'); /* ... */ console.timeEnd('carga');   // mide tiempos
console.trace();                  // muestra la pila de llamadas
console.assert(x > 0, 'x debe ser positivo');
```

6.4 El depurador de VS Code

6.4.1 Conceptos

Término	Qué es
Punto de ruptura (*breakpoint*)	Marca en una línea: la ejecución se detiene ahí
Punto de ruptura condicional	Solo se detiene si se cumple una expresión (`i === 42`)
Punto de registro (*logpoint*)	No detiene: escribe un mensaje. `console.log` sin tocar el código
Paso a paso por encima (*step over*, `F10`)	Ejecuta la línea; si es una llamada, no entra
Paso a paso por dentro (*step into*, `F11`)	Entra dentro de la función llamada
Salir (*step out*, `Shift+F11`)	Termina la función actual y vuelve
Continuar (F5)	Sigue hasta el próximo punto de ruptura
Pila de llamadas (*call stack*)	Quién ha llamado a quién hasta llegar aquí
Variables	Estado actual de locales, argumentos y globales
Inspección (*watch*)	Expresiones que quieres vigilar en cada parada
Consola de depuración	Evaluar expresiones en el contexto pausado

6.4.2 `launch.json`

`.vscode/launch.json`:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Depurar GestorTareas",
      "program": "${workspaceFolder}/src/index.js",
      "args": ["listar"],
      "console": "integratedTerminal",
      "skipFiles": ["<node_internals>/**"]
    },
    {
      "type": "node",
      "request": "launch",
      "name": "Depurar pruebas de Jest",
      "program": "${workspaceFolder}/node_modules/.bin/jest",
      "args": ["--runInBand", "--no-coverage"],
      "console": "integratedTerminal",
      "internalConsoleOptions": "neverOpen"
    }
  ]
}
```

`"skipFiles": ["<node_internals>/**"]` es importante: sin él, al hacer *step into* acabas dentro de las tripas de Node y no sabes salir.

Práctica

P3.3 — Caza de errores con el depurador (2 h)
Se entrega `buggy.js` con **tres** defectos: uno de límite (< vs <=), uno de estado (una variable que no se reinicia) y uno de tipo (comparación con == y coerción).
Para cada uno:

1. Reproduce el fallo y descríbelo (resultado esperado vs obtenido).
2. Formula tu hipótesis **por escrito, antes** de depurar.
3. Pon un punto de ruptura y localiza la línea exacta. Captura de la pila de llamadas y del panel de variables en el momento del fallo.
4. Usa al menos una vez: un punto de ruptura **condicional**, un **logpoint** y una expresión de **inspección**.
5. Corrige.
6. Escribe el caso de prueba (todavía en la tabla, sin Jest) que habría detectado el error.

Prohibido usar `console.log` en esta práctica.

7 Pruebas unitarias automáticas con Jest

7.1 Instalación

```
npm install -D jest
```

En `package.json`:

```
{
  "scripts": {
    "test": "jest",
    "test:watch": "jest --watch",
    "test:cov": "jest --coverage"
  }
}
```

Con módulos ES (`"type": "module"`) hay que arrancar Jest con el soporte experimental:

```
"test": "node --experimental-vm-modules node_modules/jest/bin/jest.js"
```

7.2 Anatomía de una prueba

```
// test/tarea.test.js
import { Tarea } from '../src/tarea.js';

describe('Tarea', () => {

  describe('constructor', () => {
    test('crea una tarea pendiente por defecto', () => {
      const t = new Tarea({ id: 1, titulo: 'Estudiar' });
      expect(t.hecha).toBe(false);
      expect(t.titulo).toBe('Estudiar');
    });

    test('lanza TypeError si el título está vacío', () => {
      expect(() => new Tarea({ id: 1, titulo: '' })).toThrow(TypeError);
    });
  });

  describe('estaVencida', () => {
    test.each([
      ['2026-01-01', '2026-02-01', true],   // fecha pasada
      ['2026-03-01', '2026-02-01', false],  // fecha futura
      ['2026-02-01', '2026-02-01', false],  // mismo día: NO vencida (límite)
    ])('límite %s comparada con %s → %s', (limite, ahora, esperado) => {
      const t = new Tarea({ id: 1, titulo: 'x', fechaLimite: limite });
      expect(t.estaVencida(new Date(ahora))).toBe(esperado);
    });
  });

});
```

7.2.1 Estructura AAA

Toda prueba bien escrita tiene tres partes:

```
test('marcar como hecha cambia el estado', () => {
  // Arrange (preparar)
  const tarea = new Tarea({ id: 1, titulo: 'Estudiar' });

  // Act (actuar)
  tarea.marcarHecha();

  // Assert (comprobar)
  expect(tarea.hecha).toBe(true);
});
```

7.3 Matchers principales

Matcher	Para qué
`toBe(x)`	Igualdad estricta (===). Para primitivos
`toEqual(x)`	Igualdad **profunda**. Para objetos y arrays
`toStrictEqual(x)`	Como `toEqual` pero también compara clases y `undefined`
`toBeTruthy()` / `toBeFalsy()`	Valores «verdaderos»/«falsos»
`toBeNull()` / `toBeUndefined()` / `toBeDefined()`	Nulos
`toContain(x)`	Un array contiene un elemento; una cadena, una subcadena
`toHaveLength(n)`	Longitud
`toMatch(/regex/)`	Coincidencia de cadena
`toThrow(Error)`	La función lanza excepción
`toBeGreaterThan(n)`, `toBeLessThanOrEqual(n)`	Comparaciones
`toBeCloseTo(n, dígitos)`	Decimales (`0.1 + 0.2 !== 0.3`)
`toHaveProperty('a.b')`	Propiedad anidada
`.not.`	Niega cualquier matcher: `expect(x).not.toBe(3)`

Ojo

`toBe` vs `toEqual` es **el** error clásico del principiante:

```
expect({a: 1}).toBe({a: 1});     // FALLA: son objetos distintos en memoria
expect({a: 1}).toEqual({a: 1});  // PASA: contenido igual
```

Y con decimales: `expect(0.1 + 0.2).toBe(0.3)` **falla**, porque en coma flotante da `0.30000000000000004`. Usa `toBeCloseTo(0.3)`.

7.4 Preparación y limpieza

```
let almacen;

beforeAll(()  => { /* una vez, antes de todas las pruebas del bloque */ });
beforeEach(() => { almacen = new Almacen('/tmp/test.json'); });  // antes de cada una
```

```
afterEach(()  => { fs.rmSync('/tmp/test.json', { force: true }); });
afterAll(()   => { /* una vez, al final */ });
```

Idea clave

Cada prueba debe ser **independiente**: si el orden de ejecución cambia, deben seguir pasando. Por eso se usa `beforeEach` para crear datos frescos, en lugar de compartir una variable entre pruebas. Las pruebas que dependen unas de otras son una fuente inagotable de dolores de cabeza.

7.5 Dobles de prueba (mocks)

Cuando lo que quieres probar depende de algo externo (el reloj, la red, el disco), lo sustituyes por un **doble**.

```
// Función simulada que registra cómo se la ha llamado
const guardarFalso = jest.fn();

almacen.guardar = guardarFalso;
servicio.añadirTarea('Estudiar');

expect(guardarFalso).toHaveBeenCalledTimes(1);
expect(guardarFalso).toHaveBeenCalledWith(
  expect.arrayContaining([expect.objectContaining({ titulo: 'Estudiar' })])
);
```

Vocabulario (Meszaros):

- **Dummy**: se pasa pero no se usa.
- **Stub**: devuelve respuestas prefabricadas.
- **Spy**: como el real, pero registra las llamadas.
- **Mock**: espera que se le llame de una forma concreta y falla si no.
- **Fake**: implementación simplificada que funciona de verdad (una base de datos en memoria).

7.6 Cobertura

```
npx jest --coverage
```

```
----------------|---------|----------|---------|---------|-------------------
File            | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
----------------|---------|----------|---------|---------|-------------------
All files       |   87.50 |    83.33 |   90.00 |   87.50 |
 tarea.js       |   95.00 |    91.66 |  100.00 |   95.00 | 42
 almacen.js     |   75.00 |    66.66 |   80.00 |   75.00 | 18-22
----------------|---------|----------|---------|---------|-------------------
```

Se puede exigir un mínimo en `package.json` o `jest.config.js`:

```
export default {
  collectCoverageFrom: ['src/**/*.js'],
  coverageThreshold: {
    global: { statements: 80, branches: 80, functions: 80, lines: 80 },
  },
};
```

Si baja del umbral, `npm test` **falla**. Es la forma de que la cobertura no se degrade con el tiempo.

Práctica

P3.4 — De la tabla de casos a las pruebas automáticas (2 h)

1. Instala y configura Jest en `gestor-tareas`.
2. Convierte en pruebas de Jest la tabla de casos que diseñaste en **P3.2** para `validarDNI`. Usa `test.each` para no repetirte.
3. Escribe pruebas para `Tarea` (constructor, `marcarHecha`, `estaVencida`) cubriendo clases de equivalencia y valores límite.
4. Escribe pruebas para `Almacen` usando un fichero temporal, con `beforeEach`/`afterEach`.
5. Usa al menos: `toBe`, `toEqual`, `toThrow`, `toContain`, `toHaveLength` y un `.not.`.
6. Ejecuta `npx jest --coverage` y pega el informe.
7. Identifica **una línea sin cubrir** y escribe la prueba que la cubre.

8 Desarrollo dirigido por pruebas (TDD)

8.1 El ciclo rojo-verde-refactor

1. **ROJO**: escribe una prueba que falle. Todavía no existe el código.
2. **VERDE**: escribe **el mínimo** código que haga pasar la prueba. Vale hacer trampas.
3. **REFACTOR**: limpia el código, con la red de la prueba protegiéndote.

Y vuelta a empezar.

8.2 Ejemplo: contador de tareas pendientes, con TDD

```
// PASO 1 — ROJO
test('devuelve 0 con la lista vacía', () => {
  expect(contarPendientes([])).toBe(0);
});
// Falla: contarPendientes no existe
```

```
// PASO 2 — VERDE (mínimo, sin vergüenza)
export function contarPendientes(tareas) {
  return 0;
}
```

```
// PASO 3 — ROJO otra vez
test('cuenta una tarea pendiente', () => {
  expect(contarPendientes([{ hecha: false }])).toBe(1);
});
```

```
// PASO 4 — VERDE de verdad
export function contarPendientes(tareas) {
  return tareas.filter(t => !t.hecha).length;
}
```

```
// PASO 5 — ROJO: caso que no habíamos pensado
test('ignora las hechas', () => {
  expect(contarPendientes([{ hecha: true }, { hecha: false }])).toBe(1);
});
// Pasa sin tocar nada. Bien: la implementación ya era correcta.
```

8.3 Por qué funciona

- Te **obliga** a pensar la interfaz antes que la implementación.
- La cobertura sale sola, no como tarea pesada posterior.
- Tienes una red de seguridad continua para refactorizar: es justo lo que vas a necesitar en UD4.
- Evita escribir código «por si acaso» que nadie usa.

Ojo

TDD no es obligatorio ni la única forma correcta de trabajar; es una técnica más. Pero para el examen: **la prueba se escribe ANTES que el código**. Ese es el rasgo que lo define.

9 Calidad del software: normas

Criterio RA3-h: *«se han interpretado normas y certificaciones»*.

9.1 ISO/IEC 25010 — Modelo de calidad del producto

Sustituye a la clásica ISO 9126. Define **ocho características** de calidad del producto:

Característica	Qué significa
Adecuación funcional	Hace lo que debe, completo y correcto
Eficiencia de desempeño	Tiempos y consumo de recursos adecuados
Compatibilidad	Coexiste e interopera con otros sistemas
Usabilidad	Fácil de aprender, de usar y protegido frente a errores del usuario
Fiabilidad	Madurez, disponibilidad, tolerancia a fallos, recuperabilidad
Seguridad	Confidencialidad, integridad, no repudio, autenticidad
Mantenibilidad	Modularidad, reusabilidad, analizabilidad, modificabilidad, **capacidad de ser probado**
Portabilidad	Adaptabilidad, instalabilidad, reemplazabilidad

Idea clave

Fíjate en que **«capacidad de ser probado» es una subcaracterística oficial de la mantenibilidad**. No es una manía de los profesores: es una propiedad del producto reconocida por la norma internacional. Un código que no se puede probar es, por definición, un código de peor calidad.

9.2 Otras normas y certificaciones que conviene nombrar

- **ISO/IEC 25000 (SQuaRE)**: la familia completa de normas de calidad del software, de la que 25010 es el modelo.
- **ISO/IEC 12207**: procesos del ciclo de vida del software.
- **ISO 9001**: gestión de la calidad de la organización (no del producto).
- **CMMI**: modelo de madurez de los procesos de una organización, en 5 niveles (inicial, gestionado, definido, gestionado cuantitativamente, en optimización).
- **ISTQB**: certificación profesional para personas que se dedican a las pruebas. La más reconocida del sector.

Práctica

P3.5 — Entrega evaluable de la unidad (evaluable)
Repositorio de `gestor-tareas` que incluya:

1. Documentación (35 %)

- `docs/plan-pruebas.md` con la estructura vista en clase.
- `docs/casos-prueba.md` u hoja de cálculo con **al menos 25 casos** documentados con los 7 campos.
- Dos informes de incidencia completos correspondientes a errores reales que hayas encontrado.

2. Pruebas automáticas (50 %)

- Batería de Jest con **al menos 30 pruebas**.
- Uso de `describe` anidados, `test.each`, `beforeEach`/`afterEach` y al menos un *mock*.
- **Cobertura ≥ 80 %** en sentencias y ramas, con umbral configurado en `jest.config.js`.
- Al menos una función desarrollada con **TDD**: entrega las dos versiones del fichero de prueba (la que falla y la que pasa) para que se vea que la prueba se escribió primero.

3. Depuración (15 %)

- `.vscode/launch.json` con dos configuraciones.
- Un documento que narre la caza de un error real: hipótesis, capturas del depurador, corrección y prueba de regresión añadida.

Idea clave

Guarda esta batería tal cual: en UD4 la vas a poner a correr automáticamente en cada `push` con GitHub Actions, y va a ser la que te permita refactorizar sin miedo.

Rúbrica

Criterio	Nivel alto	Nivel medio	Nivel bajo
Diseño de casos	Técnicas aplicadas de forma sistemática y justificada	Casos razonables sin técnica clara	Casos improvisados
Pruebas Jest	Independientes, legibles, con casos límite y mocks	Funcionan pero repetitivas	Pocas o triviales
Cobertura	≥ 80 % con umbral configurado	60-80 %	< 60 %
Documentación	Plan e incidencias completos y utilizables	Incompletos	Ausentes
Depuración	Método claro, uso avanzado del depurador	Uso básico	`console.log`

10 Resumen de la unidad

Concepto	En una línea
Verificación	¿Lo construimos **correctamente**? (contra la especificación)
Validación	¿Construimos el producto **correcto**? (contra la necesidad)
Error / defecto / fallo	Equivocación humana / consecuencia en el código / lo que ve el usuario
Caja negra	Se prueba por la interfaz, sin ver el código
Caja blanca	Se diseñan casos mirando la estructura del código
Pirámide de pruebas	Muchas unitarias, algunas de integración, pocas E2E
Prueba de regresión	Comprobar que lo que iba, sigue yendo
Clases de equivalencia	Dividir entradas en grupos y probar un representante
Valores límite	Probar las fronteras y sus vecinos
Tabla de decisión	Combinaciones de varias condiciones
Cobertura de sentencias	% de líneas ejecutadas
Cobertura de ramas	Cada decisión evaluada a V y a F
Caso de prueba	ID, precondiciones, entrada, pasos, esperado, obtenido, veredicto
Severidad vs prioridad	Daño técnico vs urgencia de negocio
Punto de ruptura	Marca donde se detiene la ejecución
Logpoint	Escribe un mensaje sin detener y sin tocar el código
Pila de llamadas	Cadena de quién llamó a quién
AAA	Arrange, Act, Assert
`toBe` vs `toEqual`	Identidad estricta vs igualdad profunda
Mock / stub / spy	Dobles que sustituyen dependencias externas
TDD	Prueba primero, código mínimo después, refactorizar al final
ISO/IEC 25010	8 características de calidad del producto software
CMMI	5 niveles de madurez de los procesos de una organización

11 Autoevaluación

1. Un programa cumple todas sus especificaciones pero el cliente dice que no le sirve. ¿Ha fallado la verificación o la validación?
2. Diseña las clases de equivalencia para una función que acepta una nota entre 0 y 10 con un decimal.
3. ¿Cuántos casos mínimos necesitas para cobertura de decisiones en una función con dos `if` independientes? ¿Y para cobertura de caminos?
4. ¿Por qué `expect(0.1 + 0.2).toBe(0.3)` falla? ¿Cómo se prueba eso bien?
5. Tienes 100 % de cobertura y el programa tiene un bug grave. ¿Es posible? Explica cómo.
6. Diferencia entre un punto de ruptura condicional y un logpoint.
7. Ordena los pasos de TDD y di qué se hace en cada uno.
8. ¿Por qué en CI se usa `npm ci` y no `npm install`?
9. Cita cuatro de las ocho características de la ISO/IEC 25010.
10. Acabas de arreglar un bug. ¿Qué te falta hacer antes de dar la tarea por cerrada?